

ASEN 5264 Final Project: GEO Reconnaissance for Space Domain Awareness

Reno Brown

Ann and H.J. Smead Aerospace Engineering Sciences

University of Colorado Boulder

Boulder, CO

reno.brown@colorado.edu

Abstract—This paper addresses the problem of geosynchronous earth orbit reconnaissance for space domain awareness. The objective is to optimally observe and rendezvous with uncharacterized space objects in order to identify adversaries among normal spacecraft and debris. Because target types are initially unknown, the problem is formulated as a partially observable Markov decision process (POMDP). Two solution strategies are evaluated: a Monte Carlo tree search using the POMCP algorithm and belief-space reinforcement learning via proximal policy optimization (PPO). The reinforcement learning approach is found to significantly outperform both the POMCP method and a random baseline, producing a high success rate for adversary identification while minimizing resource expenditure on debris.

I. INTRODUCTION

Geosynchronous Earth Orbit (GEO) describes a region of space about 36 000 km from the surface of the Earth where satellites can orbit with the same period as the Earth's rotation. This is an extremely useful quality and as a result there are several hundred operational satellites in this region. Given the importance of GEO satellites for earth observation, communications, and other purposes, Earth-bound militaries are beginning to pay more attention to this space. Recently, the Defense Innovation Unit solicited industry proposals for a "geosynchronous high-resolution optical, space-based tactical reconnaissance capability" [1]. Translated from mil-speak, this means a satellite that can image and visit—rendezvous with—targets on demand. This type of mission is also covered by the concept of "Space Domain Awareness", an area of active research at CU Boulder [2]. We can imagine a situation where adversary satellites are in GEO, but difficult to distinguish from normal satellites or in-operational debris. The question of how to optimally observe and visit a set of unknown targets in order to identify adversaries motivates this project.¹

II. BACKGROUND AND RELATED WORK

Deciding how to optimally visit multiple targets in orbit is a problem with several approaches already documented in literature. Mostly, these center around removing debris, such as the 2026 preprint "Optimal Multi-Debris Mission Planning

in LEO: A Deep Reinforcement Learning Approach with Co-Elliptic Transfers and Refueling" [3]. It formulates an Markov Decision Process (MDP) with $N = 50$ debris targets to be visited in low Earth orbit (LEO), and an agent with the ability to rendezvous with debris or return to be refueled. They used several techniques that can be applied to our problem, namely action masking and Masked PPO (Proximal Policy Optimization) Reinforcement learning [4]. The latter solution approach was found to be much more effective than Monte Carlo Tree Search (MCTS) for this type of problem. Another approach described in literature is to formulate it as a Traveling Salesman Problem (TSP) combinatorial decision [5]. This is a very well-studied problem, and the authors in this case found a near-optimal solution using a gradient optimizer. However, this is unsuitable for problems that involve uncertainty, like the one we aim to solve. Following [3] but formulating our problem as a POMDP instead is much more promising. Yet another interesting POMDP approach is used in "Optimal Planning and Scheduling for Satellite Servicing Under Partial Observability" [6]. This paper describes a problem where a servicing agent with no refueling capabilities needs to visit and service satellites in both medium Earth orbits (MEO) and GEO. Unlike other papers describing optimal orbital servicing, it introduces uncertainty in the health of target spacecraft. The agent can inspect, maintain, or travel to a given satellite. It models satellite degradation over time, with transition and observation dynamics reminiscent of the much-simpler Crying Baby Problem [7]. The paper describes mixing fully observable states such as orbital velocity and position with the partially observable health state to create a Mixed Observable Markov Decision Process (MOMDP). The SARSOP [8] algorithm is then used to solve, with satisfactory results. While our problem involves much different observation and reward structure, their treatment of mixed-observability is very informative.

III. PROBLEM FORMULATION

For this problem to be tractable, we must make some assumptions in our problem formulation. First, we will only consider a small number of targets that can be visited (11 to begin with). These targets are all assumed to lie perfectly in the GEO circle (zero inclination, zero eccentricity, and semimajor axis of exactly 42 164 km). These assumptions

¹Note the motivation has changed from what was discussed in the project proposal, but the underlying problem and approach is very similar. This new motivation makes more sense for a problem with uncertainty, i.e. a POMDP.

work very well for many operational GEO satellites, which tend to have inclination within $\pm 0.15^\circ$ and eccentricity of ≈ 0 [9]. Without continued station-keeping, however, debris or in-operational satellites would not lie perfectly in this circle. Keeping the problem in the 2D plane removes the need to include inclination correction in the already large action space. Transfer times can be calculated simply and deterministically from a discrete Δv phasing burn, which forms part of the action (along with the index of the target).

The following process is used to calculate transfer time based on the selected phasing Δv :

First, determine the phasing orbit velocity v_p , semi-major axis a_p , and relative angular velocity ω_{rel} :

$$v_p = v_{\text{circ}}(a_{\text{GEO}}) + \Delta v \quad (1)$$

$$a_p = \left(\frac{2}{a_{\text{GEO}}} - \frac{v_p^2}{\mu_{\text{EARTH}}} \right)^{-1} \quad (2)$$

$$\omega_{\text{rel}} = n_{\text{circ}}(a_p) - n_{\text{GEO}} \quad (3)$$

where $v_{\text{circ}}(a) = \sqrt{\mu_{\text{EARTH}}/a}$ and $n_{\text{circ}}(a) = \sqrt{\mu_{\text{EARTH}}/a^3}$.

Next, determine the required angular displacement $\Delta\lambda$ and the transfer time t_{tr} by conditionally accounting for the drift direction:

$$\Delta\lambda = \begin{cases} (\lambda_{\text{to}} - \lambda_{\text{from}}) \pmod{2\pi} & \text{if } \Delta v < 0 \\ (\lambda_{\text{from}} - \lambda_{\text{to}}) \pmod{2\pi} & \text{if } \Delta v \geq 0 \end{cases} \quad (4)$$

$$t_{\text{tr}} = \begin{cases} \infty & \text{if } a_p \leq 0 \text{ or } \Delta v \cdot \omega_{\text{rel}} \geq 0 \\ 0 & \text{if } \Delta\lambda = 0 \\ \left| \frac{\Delta\lambda}{\omega_{\text{rel}}} \right| & \text{otherwise} \end{cases} \quad (5)$$

The rewards are set in a way to guide an agent towards finding adversaries among the targets. Visiting an adversary produces the highest positive reward of +20, representing a successful rendezvous and close-up imaging of a previously hidden target. Visiting a normal operational satellite (Nominal type) produces a smaller +2 reward, since close-up imaging of a commercial satellite could still be useful for its operators (or to confirm that it has not been tampered with somehow). Visiting a Debris produces a -20 reward, to punish the wasted fuel, time and effort imaging a derelict satellite or debris. If the agent visits only debris, it has effectively been "fooled" by the still-hidden adversaries. The type of each target is the only state element unknown to the agent. Visiting a target generates a perfect observation of its type. The agent can also choose to observe remotely, which returns an imperfect observation based on the inverse square of its distance from the target. The parameter d_0 is chosen as 50 000 km, representing the distance where the observation will be 50% likely to return the correct type. The observation accuracy function is shown for a range of longitude differences in Figure 1. The line at α_{min} indicates when an observation is no better than random chance (1/3 probability). We can see that for nearby targets, within about 60° of longitude, observations are genuinely useful. This simple observation model leaves much out, but

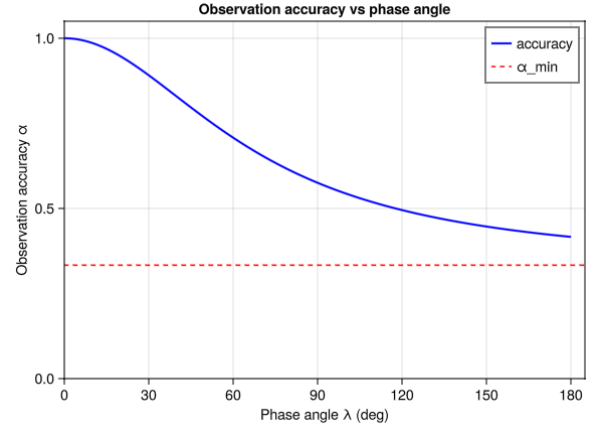


Fig. 1. Observation Accuracy Function

at least prevents accurate observations of extremely distant targets and encourages observing close-by targets.

The full formulation as a Partially Observable Markov Decision Process (POMDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, T, R, O, \mathcal{Z}, \gamma)$ is:

- $\mathcal{S}$ is the state space. The state is defined as $s = (t, \lambda, \Delta v, V, \tau)$, where $t \in \mathbb{R}^+$ is the time remaining, $\lambda \in [0, 2\pi)$ is the servicer longitude, $\Delta v \in \mathbb{R}^+$ is the remaining velocity budget, $V \in \{0, 1\}^N$ is the boolean visitation BitVector for the N targets, and $\tau \in \{\text{Debris}, \text{Nominal}, \text{Adversary}\}^N$ represents the true, unobservable target types.
- $\mathcal{A}$ is the action space, consisting of discrete sets $\mathcal{A}_{\text{phase}} \cup \mathcal{A}_{\text{obs}}$. Phase actions specify a target index i and a discrete phasing burn Δv_{phase} . Observe actions specify a target index i to observe from the current position.
- $T(s' | s, a)$ is the transition model, giving the probability of transitioning to state s' after taking action a in state s . The dynamics are defined deterministically. For an observe action, $t' = t - t_{\text{obs_cost}}$, while all other state variables remain constant. For a phase action, $t' = t - t_{\text{tr}}(a)$, $\lambda' = \lambda_i$, $\Delta v' = \Delta v - 2|\Delta v_{\text{phase}}|$, and $V'_i = 1$, assuming budget constraints hold. The transfer time $t_{\text{tr}}(a)$ is calculated via Eqs. 1-5
- $R(s, a)$ is the reward function, defined as 0 for all observe actions. For phase actions, the reward maps directly to the true underlying type of the visited target τ_i , executing $R_{\text{type}}(\tau_i) \in \{-20.0, 2.0, 20.0\}$.
- $\mathcal{Z}$ is the observation space. An observation is defined as $z = (i, \hat{\tau}_i, \lambda, V)$, where $\hat{\tau}_i$ is the perceived target type and the remaining variables are fully observable state components.
- $O(z | s', a)$ is the observation model. For phase actions, the target type is fully observable with probability 1. For observe actions, the probability of a correct classification ($\hat{\tau}_i = \tau'_i$) is:

$$\alpha(d) = \alpha_{\text{min}} + \frac{1 - \alpha_{\text{min}}}{1 + (d/d_0)^2} \quad (6)$$

where d is the shortest orbital arc distance between the agent and the target. The observation likelihood is then given by:

$$P(z | \tau, d) = \begin{cases} \alpha(d) & \text{if } z = \tau \\ \frac{1 - \alpha(d)}{2} & \text{if } z \neq \tau \end{cases} \quad (7)$$

Incorrect classifications are distributed uniformly across remaining types. All other state elements $(t, \lambda, \Delta v, V)$ are fully observable.

- γ is the discount factor, set to 1.0 (undiscounted finite-horizon).

This POMDP was implemented in Julia using the `POMDPs.jl` ecosystem. The model `GEOReconPOMDP` was created as a subtype of `POMDP`, with the custom types `GEOState`, `GEOAction`, `TypeObservation` containing the state, action and observation elements. Important parameters and their values are summarized in table I below.

TABLE I
GEO RECON POMDP PARAMETERS

Parameter	Value	Unit
Target longitudes λ_t	11 points in $[0, 2\pi)$	rad
Priors (D, N, A)	$[0.5, 0.25, 0.25]$	—
Rewards (D, N, A)	$\{-20.0, 2.0, 20.0\}$	—
Time budget	120	days
Velocity budget	0.5	km/s
Discrete phasing levels	5	—
Maximum phasing burn	0.1	km/s
Observation time cost	6 hours	hrs
Minimum observation accuracy	1/3	—
Half-accuracy distance d_0	50,000	km
Discount factor γ	1.0	—

The Δv phasing burn is discretized into 5 levels from 0 km/s to 0.1 km/s. The targets are initialized in 11 out of 12 equally spaced longitude slots, with the agent occupying the $\lambda = 0^\circ$ slot. An example arrangement with $\tau = [\text{Debris}, \text{Nominal}, \text{Nominal}, \text{Adversary}, \text{Adversary}, \text{Nominal}, \text{Adversary}, \text{Debris}, \text{Debris}, \text{Debris}, \text{Debris}]$ is shown in Figure 2. Combined with the selection of 11 targets, and the possibility of going either “east” or “west” to a target, there are a total of 121 possible actions to begin with. The state space, on the other hand, is a mixture of discrete and continuous. The scale of this problem suggests that an online planner would be preferable to an offline approach. We first attempt to use the POMCP-UCT algorithm to solve this problem. Approach details follow below.

IV. SOLUTION APPROACH

A. POMCP

The Monte-Carlo tree search (MCTS) algorithm is a robust approach to solving Markov Decision Problems (MDP). The 2010 paper “Monte-Carlo Planning in Large POMDPs” [10] applies this approach to partially observable problems. To do this, they design the algorithm to build a search tree over histories rather than states. Algorithm 1 below is reproduced from [10].

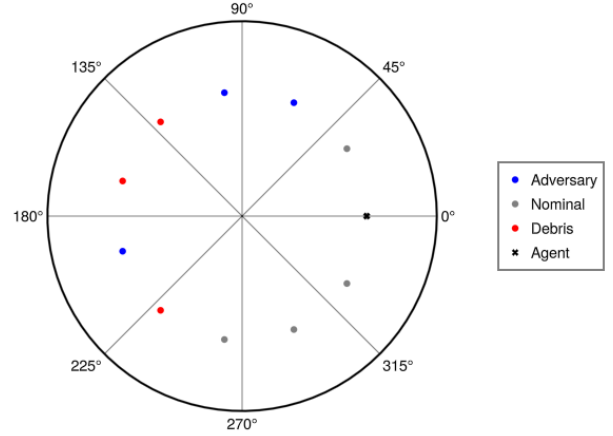


Fig. 2. Example initial τ for $N = 11$

We specifically used the implementation of POMCP in `BasicPOMCP.jl`. In order to use this algorithm, we need to define a rollout policy and also implement problem-specific features such as action masking. Action masking refers to eliminating infeasible actions from the action space in order to simplify the problem. For the GEO recon POMDP problem, we want to mask actions such as re-visiting already visited targets, observing already identified targets, or phase actions that violate the Δv or time constraints. This was achieved by placing conditional statements in the `POMDPs.actions` method definitions for this problem. A simple heuristic deterministic rollout policy is used that phases to any target observed as an Adversary. This policy is described in algorithm 2 below. A weighted particle filter with 1000 particles was used to update the beliefs. The UCB exploration constant c was chosen as 20, using the rule of thumb that it should be half the difference between max and min reward. These and other important parameters are summarized in Table II below.

Algorithm 2 Heuristic Rollout Policy

Given observation o of target i :
if $o = \text{Adversary}$ **and** i is unvisited **then**
 Phase to i
else
 Observe next unvisited target
end if

TABLE II
POMCP SOLVER CONFIGURATION PARAMETERS

Parameter	Value/Type
Number of particles	1000
Belief updater	Weighted Particle Filter
Maximum search depth	50
Exploration constant	20.0
Tree queries per step	5000
Value estimator	Rollout Estimator

Algorithm 1 Partially Observable Monte-Carlo Planning

```

procedure SEARCH( $h$ )
  repeat
    if  $h = \text{empty}$  then
       $s \sim \mathcal{I}$ 
    else
       $s \sim B(h)$ 
    end if
    SIMULATE( $s, h, 0$ )
  until TIMEOUT
  return  $\arg \max_b V(hb)$ 
end procedure
procedure ROLLOUT( $s, h, \text{depth}$ )
  if  $\gamma^{\text{depth}} < \epsilon$  then
    return 0
  end if
   $a \sim \pi_{\text{rollout}}(h, \cdot)$ 
   $(s', o, r) \sim \mathcal{G}(s, a)$ 
  return  $r + \gamma \cdot \text{ROLLOUT}(s', hao, \text{depth} + 1)$ 
end procedure
procedure SIMULATE( $s, h, \text{depth}$ )
  if  $\gamma^{\text{depth}} < \epsilon$  then
    return 0
  end if
  if  $h \notin T$  then
    for all  $a \in \mathcal{A}$  do
       $T(ha) \leftarrow (N_{\text{init}}(ha), V_{\text{init}}(ha), \emptyset)$ 
    end for
    return ROLLOUT( $s, h, \text{depth}$ )
  end if
   $a \leftarrow \arg \max_b V(hb) + c \sqrt{\frac{\log N(h)}{N(hb)}}$ 
   $(s', o, r) \sim \mathcal{G}(s, a)$ 
   $R \leftarrow r + \gamma \cdot \text{SIMULATE}(s', hao, \text{depth} + 1)$ 
   $B(h) \leftarrow B(h) \cup \{s\}$ 
   $N(h) \leftarrow N(h) + 1$ 
   $N(ha) \leftarrow N(ha) + 1$ 
   $V(ha) \leftarrow V(ha) + \frac{R - V(ha)}{N(ha)}$ 
  return  $R$ 
end procedure

```

B. Belief-Space MDP Reinforcement Learning with PPO

To find a better policy (higher total reward and more adversaries visited), we turn to reinforcement learning. As with POMCP, RL strategies for POMDPs are built upon algorithms developed for MDPs. Given its successful use in [3], the PPO technique will be used. The PPO algorithm is described in [4]. Its main innovation is to gradient optimize a clipped "surrogate" objective (Eq. 7). To adapt PPO to work on a POMDP, we must account for the hidden state by using the belief as the state for an MDP. This is known as a belief-space MDP. For this problem, this means the MDP state includes an $N \times 3$ array of type probabilities where N is the number of targets. The remaining elements of the state are $(t, \lambda, \Delta v, V)$ which continue to be perfectly

known. For suitable training, this state is flattened into a $3(11)+3+1(11) = 47$ -dimension **Vector{Float32}**, with each element normalized to the range of $[0, 1]$ (including Δv and t divided by their respective budget). The probabilities are initialized as the prior $[0.5, 0.25, 0.25]$ and updated outside of the neural network using a particle filter. Similar to the POMCP, we use a weighted particle filter with 1000 particles.

Actions are masked using the already defined `POMDPs.actions` method. The well-known Python RL ecosystem Stable Baselines3 [11] is used to implement the PPO with masking. The same `GeoReconPOMDP` model is used, but passed to Python using the Julia package `PythonCall.jl`. Generalized Advantage Estimate is used inside the SB3 policy, which ensures that there is credit attribution for good observation actions [12]. The neural network is set up with two hidden layers with 256 nodes each. The batch size is also set at 256. Total steps was set to 500,000, which led to a training time of about 1 hour on mid-spec hardware. These and other parameters are summarized in Table III below.

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad (7)$$

TABLE III
PPO CONFIGURATION PARAMETERS

Parameter	Value/Type
Algorithm	MaskablePPO (sb3-contrib)
Policy type	MlpPolicy
Network architecture	[256, 256]
Total timesteps	500,000
Rollout buffer size (n_steps)	2048
Batch size	256
Optimization epochs (n_epochs)	4
Learning rate	3×10^{-4}
Entropy coefficient (ent_coef)	0.02
Clip range	0.2

V. RESULTS

TABLE IV
GEO RECON POMDP RESULTS

Metric	Baseline (Random)	POMCP	PPO
Return $\pm$ SEM	-16.88 $\pm$ 0.94	-7.32 $\pm$ 0.78	35.12 $\pm$ 1.10
All Found (%)	7.5	0.9	74.7
Adv. Found	1.40 / 2.76	0.80 / 2.73	2.41 / 2.74
Debris Visited	2.38	1.25	0.82
Obs. Actions	94.6	10.3	133.8
Δv (km/s)	0.481	0.201	0.471

A. POMCP

Early results from the POMCP planner tended towards aggressive phasing, which nearly guaranteed large negative rewards from the debris-favoring distribution. After much tweaking to both the model and the solver, the parameters

reflected in Tables I and II produced a POMCP planner that would reliably observe as well as phase. Results from 1000 Monte Carlo runs with this POMCP planner are shown in Table IV. The same amount of runs were done using a purely random policy as a baseline to compare with. We find that the POMCP planner reliably beats pure random, by about 7 undiscounted total return, but remains negative overall. Other metrics to note are the percentage of episodes where all adversaries were found, and the average number of adversaries found overall. The pure random policy nets more episodes with all adversaries found, due to its more aggressive nature. The POMCP planner does worse on these metrics (0.9% episodes with all found, and 0.8 adv. found avg per episode), because it does not have as many chances to "get lucky". In this way, it ends up with a higher total reward because it does not blindly phase into Debris as often (1.25 avg Debris visited per episode vs 2.38 for random baseline). The lower delta-V might be taken as an advantage of this approach, but is perhaps better framed as the algorithm being too timid and failing to make the most of its action space. The performance overall is disappointing, and motivates moving to the reinforcement learning approach described in Section IV.B.

B. PPO

After implementing the Masked PPO algorithm, only a few training runs were necessary to see vast improvements over the POMCP and baseline policies. After some iteration and tweaking, the parameters were set to what they are in Table III. The initial training took more than an hour, but the Monte Carlo evaluation of 1000 runs with the trained model was completed much more quickly than for the POMCP planner. The tensorboard logging feature of SB3 was used to generate plots of per-episode reward and the neural network loss, seen in Figure 3 and 4 respectively. We can see a reward curve that trends upwards very nicely, before dipping down slightly in the final steps of the training. This is not surprising behavior for reinforcement learning, and if training were extended further it would likely bounce back. The loss plot in Figure 4 is not as smooth, but still shows loss trending down from its early spikes. The fact that the loss is not yet approaching zero indicates that the model could use benefit from a longer training run.

The results of the Monte Carlo runs in Table IV are similarly encouraging, as it beats both the baseline and POMCP policy in all of the most important metrics. The total rewards is not only positive but much higher, and is much closer to the theoretical optimal upper bound of ≈ 55 ($20 \times$ avg. # of adversaries). Nearly 75% of episodes result in the agent visiting all adversaries, and the average number of debris visited is less than 1 per episode. The one strange result is the shockingly high number of observe actions per episode, which is even higher than the random baseline. Upon investigation, this seems to be the result of the agent repeating meaningless observe actions after it has successfully phased to its preferred targets, until the episode terminates at the end of time. Since each observe action takes 6 hours, this means the

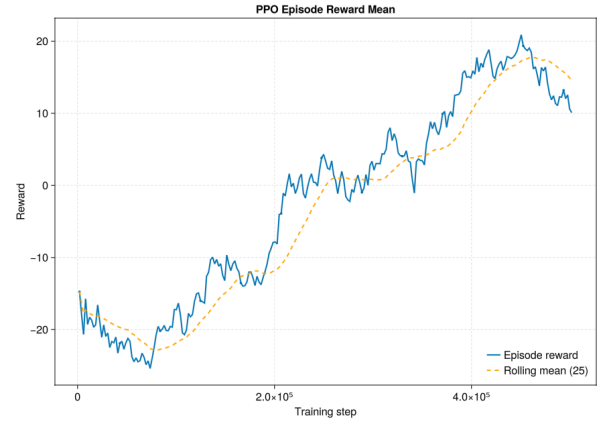


Fig. 3. PPO Episode Reward

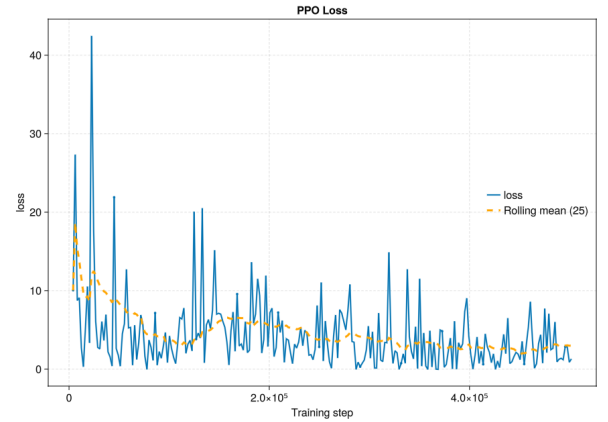


Fig. 4. PPO Neural Network Loss

agent is spending ≈ 33 days just observing. The implication for the overarching POMDP is that the time budget of 120 days is not a driving constraint for this agent. If this observe-spamming was undesirable, it could also be punished with a slight negative reward and a neutral "wait" action could be introduced as well. However, it is not clear that this is the case—what else would an idle reconnaissance satellite do but continue to passively observe?

VI. CONCLUSION

Overall, belief-space MDP reinforcement learning using PPO proved to be a high-performing solution for this problem. It was able to achieve a satisfactory level of adversary visitation, whereas the POMCP planner struggled to do much better than the pure random baseline. The loss and average episode reward do not flatline, indicating that there could be further performance gains to be had with longer neural networking training times and possibly with additional tuning. This could be the basis for future work. More interesting, however, would be to enhance the underlying POMDP model to better reflect a realistic scenario. The underlying dynamics are sound, but the perfect circular and planar GEO ring assumption leaves a lot

to be desired. If the agents are distributed with orbital elements of actual GEO objects, the agent would need to take into account costly inclination changes for its phasing decisions. Additionally, it would be interesting to use an observation model that took in approximately known orbital elements of targets as well as distance-based visual observations. In reality, it would be very easy to tell a nominal operational spacecraft apart from debris, since the operational spacecraft would be carefully maintaining its orbit through station-keeping. The observation model could take this into account while still maintaining a belief uncertainty over whether the adversary spacecraft are "hiding" as debris or a nominal spacecraft. A model to this end might make for useful progress in the domain of Space Domain Awareness, unlike the simple model detailed here. Still, the results of this paper are a not-insignificant step in that direction.

ACKNOWLEDGMENT

- **The sole author contributed to all content within**
- **All algorithms described within were implemented "off-the-shelf"**
- **The authors grant permission for this report to be posted publicly**

REFERENCES

- [1] "Pentagon Accelerates GHOST-R Hybrid Surveillance Program Following New START Expiration – SatNews." [Online]. Available: <https://satnews.com/2026/03/03/pentagon-accelerates-ghost-r-hybrid-surveillance-program-following-new-start-expiration/>
- [2] "Space Domain Awareness | Center for National Security Initiatives | University of Colorado Boulder." [Online]. Available: <https://www.colorado.edu/center/ncsi/space-domain-awareness>
- [3] A. Bandyopadhyay and G. Waxenegger-Wilfing, "Optimal Multi-Debris Mission Planning in LEO: A Deep Reinforcement Learning Approach with Co-Elliptic Transfers and Refueling," Feb. 2026, arXiv:2602.17685. [Online]. Available: <http://arxiv.org/abs/2602.17685>
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," Aug. 2017, arXiv:1707.06347. [Online]. Available: <http://arxiv.org/abs/1707.06347>
- [5] L. Hou and A. Misra, "Traveling Salesman Problem of optimal debris removal sequence using non-population gradient search," *Acta Astronautica*, vol. 215, pp. 373–386, Feb. 2024. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0094576523006550>
- [6] S. Patnala, V. Mousseau, and A. Abdin, "Optimal Planning and Scheduling for Satellite Servicing Under Partial Observability," in *AIAA SCITECH 2026 Forum*. Orlando, FL: American Institute of Aeronautics and Astronautics, Jan. 2026. [Online]. Available: <https://arc.aiaa.org/doi/10.2514/6.2026-2634>
- [7] "Defining a POMDP · POMDPs.jl." [Online]. Available: https://juliapomdp.github.io/POMDPs.jl/stable/example_defining_problems/
- [8] H. Kurniawati, D. Hsu, and W. S. Lee, "SARSOP: Efficient Point-Based POMDP Planning by Approximating Optimally Reachable Belief Spaces," in *Robotics: Science and Systems IV*. MIT Press, 2009, pp. 65–72. [Online]. Available: <https://ieeexplore.ieee.org/document/6284837>
- [9] "Inclined orbit satellite operation." [Online]. Available: <https://www.satsig.net/satellite/inclined-orbit-operation.htm>
- [10] D. Silver and J. Veness, "_USMonte-Carlo planning in large POMDPs," *_USMIT web domain*, 2010. [Online]. Available: <https://dspace.mit.edu/handle/1721.1/100395>
- [11] "DLR-RM/stable-baselines3," Apr. 2026, original-date: 2020-05-05T05:52:26Z. [Online]. Available: <https://github.com/DLR-RM/stable-baselines3>
- [12] "Proximal Policy Optimization — Spinning Up documentation." [Online]. Available: <https://spinningup.openai.com/en/latest/algorithms/ppo.html>